

4.2 Simulated Annealing

Simulated Annealing, SA.

4.2.1 Taxonomy

Simulated Annealing is a global optimization algorithm that belongs to the field of Stochastic Optimization and Metaheuristics. Simulated Annealing is an adaptation of the Metropolis-Hastings Monte Carlo algorithm and is used in function optimization. Like the Genetic Algorithm (Section 3.2), it provides a basis for a large variety of extensions and specialization's of the general method not limited to Parallel Simulated Annealing, Fast Simulated Annealing, and Adaptive Simulated Annealing.

4.2.2 Inspiration

Simulated Annealing is inspired by the process of annealing in metallurgy. In this natural process a material is heated and slowly cooled under controlled conditions to increase the size of the crystals in the material and reduce their defects. This has the effect of improving the strength and durability of the material. The heat increases the energy of the atoms allowing them to move freely, and the slow cooling schedule allows a new low-energy configuration to be discovered and exploited.

4.2.3 Metaphor

Each configuration of a solution in the search space represents a different internal energy of the system. Heating the system results in a relaxation of the acceptance criteria of the samples taken from the search space. As the system is cooled, the acceptance criteria of samples is narrowed to focus on improving movements. Once the system has cooled, the configuration will represent a sample at or close to a global optimum.

4.2.4 Strategy

The information processing objective of the technique is to locate the minimum cost configuration in the search space. The algorithms plan of action is to probabilistically re-sample the problem space where the acceptance of new samples into the currently held sample is managed by a probabilistic function that becomes more discerning of the cost of samples it accepts over the execution time of the algorithm. This probabilistic decision is based on the Metropolis-Hastings algorithm for simulating samples from a thermodynamic system.

4.2.5 Procedure

Algorithm 4.2.1 provides a pseudocode listing of the main Simulated Annealing algorithm for minimizing a cost function.

Algorithm 4.2.1: Pseudocode for Simulated Annealing.

Input: ProblemSize, $iterations_{max}$, $temp_{max}$
Output: S_{best}

```

1  $S_{current} \leftarrow \text{CreateInitialSolution}(\text{ProblemSize});$ 
2  $S_{best} \leftarrow S_{current};$ 
3 for  $i = 1$  to  $iterations_{max}$  do
4    $S_i \leftarrow \text{CreateNeighborSolution}(S_{current});$ 
5    $temp_{curr} \leftarrow \text{CalculateTemperature}(i, temp_{max});$ 
6   if  $\text{Cost}(S_i) \leq \text{Cost}(S_{current})$  then
7      $S_{current} \leftarrow S_i;$ 
8     if  $\text{Cost}(S_i) \leq \text{Cost}(S_{best})$  then
9        $S_{best} \leftarrow S_i;$ 
10    end
11  else if  $\text{Exp}(\frac{\text{Cost}(S_{current}) - \text{Cost}(S_i)}{temp_{curr}}) > \text{Rand}()$  then
12     $S_{current} \leftarrow S_i;$ 
13  end
14 end
15 return  $S_{best};$ 

```

4.2.6 Heuristics

- Simulated Annealing was designed for use with combinatorial optimization problems, although it has been adapted for continuous function optimization problems.
- The convergence proof suggests that with a long enough cooling period, the system will always converge to the global optimum. The downside of this theoretical finding is that the number of samples taken for optimum convergence to occur on some problems may be more than a complete enumeration of the search space.
- Performance improvements can be given with the selection of a candidate move generation scheme (neighborhood) that is less likely to generate candidates of significantly higher cost.
- Restarting the cooling schedule using the best found solution so far can lead to an improved outcome on some problems.
- A common acceptance method is to always accept improving solutions and accept worse solutions with a probability of $P(\text{accept}) \leftarrow$

$\exp(\frac{e-e'}{T})$, where T is the current temperature, e is the energy (or cost) of the current solution and e' is the energy of a candidate solution being considered.

- The size of the neighborhood considered in generating candidate solutions may also change over time or be influenced by the temperature, starting initially broad and narrowing with the execution of the algorithm.
- A problem specific heuristic method can be used to provide the starting point for the search.

4.2.7 Code Listing

Listing 4.1 provides an example of the Simulated Annealing algorithm implemented in the Ruby Programming Language. The algorithm is applied to the Berlin52 instance of the Traveling Salesman Problem (TSP), taken from the TSPLIB. The problem seeks a permutation of the order to visit cities (called a tour) that minimizes the total distance traveled. The optimal tour distance for Berlin52 instance is 7542 units.

The algorithm implementation uses a two-opt procedure for the neighborhood function and the classical $P(accept) \leftarrow \exp(\frac{e-e'}{T})$ as the acceptance function. A simple linear cooling regime is used with a large initial temperature which is decreased each iteration.

```

1 def euc_2d(c1, c2)
2   Math.sqrt((c1[0] - c2[0])**2.0 + (c1[1] - c2[1])**2.0).round
3 end
4
5 def cost(permutation, cities)
6   distance = 0
7   permutation.each_with_index do |c1, i|
8     c2 = (i==permutation.size-1) ? permutation[0] : permutation[i+1]
9     distance += euc_2d(cities[c1], cities[c2])
10  end
11  return distance
12 end
13
14 def random_permutation(cities)
15   perm = Array.new(cities.size){|i| i}
16   perm.each_index do |i|
17     r = rand(perm.size-i) + i
18     perm[r], perm[i] = perm[i], perm[r]
19   end
20   return perm
21 end
22
23 def stochastic_two_opt!(perm)
24   c1, c2 = rand(perm.size), rand(perm.size)
25   exclude = [c1]
26   exclude << ((c1==0) ? perm.size-1 : c1-1)

```

```

27   exclude << ((c1==perm.size-1) ? 0 : c1+1)
28   c2 = rand(perm.size) while exclude.include?(c2)
29   c1, c2 = c2, c1 if c2 < c1
30   perm[c1...c2] = perm[c1...c2].reverse
31   return perm
32 end
33
34 def create_neighbor(current, cities)
35   candidate = {}
36   candidate[:vector] = Array.new(current[:vector])
37   stochastic_two_opt!(candidate[:vector])
38   candidate[:cost] = cost(candidate[:vector], cities)
39   return candidate
40 end
41
42 def should_accept?(candidate, current, temp)
43   return true if candidate[:cost] <= current[:cost]
44   return Math.exp((current[:cost] - candidate[:cost]) / temp) > rand()
45 end
46
47 def search(cities, max_iter, max_temp, temp_change)
48   current = {:vector=>random_permutation(cities)}
49   current[:cost] = cost(current[:vector], cities)
50   temp, best = max_temp, current
51   max_iter.times do |iter|
52     candidate = create_neighbor(current, cities)
53     temp = temp * temp_change
54     current = candidate if should_accept?(candidate, current, temp)
55     best = candidate if candidate[:cost] < best[:cost]
56     if (iter+1).modulo(10) == 0
57       puts " > iteration #{(iter+1)}, temp=#{temp}, best=#{best[:cost]}"
58     end
59   end
60   return best
61 end
62
63 if __FILE__ == $0
64   # problem configuration
65   berlin52 = [[565,575],[25,185],[345,750],[945,685],[845,655],
66     [880,660],[25,230],[525,1000],[580,1175],[650,1130],[1605,620],
67     [1220,580],[1465,200],[1530,5],[845,680],[725,370],[145,665],
68     [415,635],[510,875],[560,365],[300,465],[520,585],[480,415],
69     [835,625],[975,580],[1215,245],[1320,315],[1250,400],[660,180],
70     [410,250],[420,555],[575,665],[1150,1160],[700,580],[685,595],
71     [685,610],[770,610],[795,645],[720,635],[760,650],[475,960],
72     [95,260],[875,920],[700,500],[555,815],[830,485],[1170,65],
73     [830,610],[605,625],[595,360],[1340,725],[1740,245]]
74   # algorithm configuration
75   max_iterations = 2000
76   max_temp = 100000.0
77   temp_change = 0.98
78   # execute the algorithm
79   best = search(berlin52, max_iterations, max_temp, temp_change)
80   puts "Done. Best Solution: c=#{best[:cost]}, v=#{best[:vector].inspect}"
81 end

```

Listing 4.1: Simulated Annealing in Ruby

4.2.8 References

Primary Sources

Simulated Annealing is credited to Kirkpatrick, Gelatt, and Vecchi in 1983 [5]. Granville, Krivanek, and Rasson provided the proof for convergence for Simulated Annealing in 1994 [2]. There were a number of early studies and application papers such as Kirkpatrick’s investigation into the TSP and minimum cut problems [4], and a study by Vecchi and Kirkpatrick on Simulated Annealing applied to the global wiring problem [7].

Learn More

There are many excellent reviews of Simulated Annealing, not limited to the review by Ingber that describes improved methods such as Adaptive Simulated Annealing, Simulated Quenching, and hybrid methods [3]. There are books dedicated to Simulated Annealing, applications and variations. Two examples of good texts include “Simulated Annealing: Theory and Applications” by Laarhoven and Aarts [6] that provides an introduction to the technique and applications, and “Simulated Annealing: Parallelization Techniques” by Robert Azencott [1] that focuses on the theory and applications of parallel methods for Simulated Annealing.

4.2.9 Bibliography

- [1] R. Azencott. *Simulated annealing: parallelization techniques*. Wiley, 1992.
- [2] V. Granville, M. Krivanek, and J-P. Rasson. Simulated annealing: A proof of convergence. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(6):652–656, 1994.
- [3] L. Ingber. Simulated annealing: Practice versus theory. *Math. Comput. Modelling*, 18:29–57, 1993.
- [4] S. Kirkpatrick. Optimization by simulated annealing: Quantitative studies. *Journal of Statistical Physics*, 34:975–986, 1983.
- [5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [6] P. J. M. van Laarhoven and E. H. L. Aarts. *Simulated Annealing: Theory and Applications*. Springer, 1988.